Singly Linked List – Insert At A Given Position 2 –Space Complexity and Algorithm.

***Program*: –**

```
#include <iostream>
#include <stack>
using namespace std;

typedef struct ListNode
{
    int data;
    struct ListNode *next;
} Node;

Node *head;

void createEmptyList(Node **head)
{
    *head = nullptr;
    cout << "Empty list created!!!" << endl;
}

void insertAtBeginning(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    if (*head == nullptr)
    {
        newNode->next = nullptr;
        *head = newNode;
```

```
    }
    else
    {
        newNode->next = *head;
        *head = newNode;
    }
}
void insertAtEnd(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    newNode->next = nullptr;
    if (*head == nullptr)
    {
        *head = newNode;
    }
    else
    {
        Node *current = *head;
        while (current->next != nullptr)
        {
            current = current->next;
        }
        current->next = newNode;
    }
}

void insertAtPosition(Node **head, int data, int position)
{
    Node *newNode = (Node *)malloc(sizeof(Node));

    if (newNode == nullptr)
    {
```

```
        cout << "Error: Memory allocation failed." << endl;
        return;
    }

    newNode->data = data;

    if (position == 1)
    {
        newNode->next = *head;
        *head = newNode;
        return;
    }

    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }

    if (position < 1)
    {
        cout << "Error: Invalid position." << endl;
        return;
    }
    else
    {
        Node *current = *head;
        int i = 1;
        while (i < position - 1 && current != nullptr)
        {
            current = current->next;
            i++;
        }
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }
```

```
        else
        {
            newNode->next = current->next;
            current->next = newNode;
        }
    }
}

void insertAtPosition2(Node **head, int data, int position)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    if (position == 0)
    {
        newNode->next = *head;
        *head = newNode;
        return;
    }
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }

    if (position < 0)
    {
        cout << "Error: Invalid position." << endl;
        return;
    }
    else
    {
        Node *current = *head;
```

```
        int i = 0;
        while (i < position - 1 && current != nullptr)
        {
            current = current->next;
            i++;
        }
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }

        else
        {
            newNode->next = current->next;
            current->next = newNode;
        }
    }
}


int main()
{
   createEmptyList(&head);
    while (1)
    {

        cout << "1. Insert at the beginning" << endl;
        cout << "2. Insert at the end" << endl;
        cout << "3. Insert at a position,indexing node from 1 to N" << endl;
        cout << "4. Insert at a position,indexing node from 0 to N-1" << endl;

        int choice;
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice)
        {
        case 1:
```

```
    {
        int data;
        cout << "Enter the data: ";
        cin >> data;
        insertAtBeginning(&head, data);
        break;
    }
  case 2:
  {
        int data;
        cout << "Enter the data: ";
        cin >> data;
        insertAtEnd(&head, data);
        break;
  }
 case 3:
    {
        int data,position;
        cout << "Enter the data: ";
        cin >> data;
        cout << "Enter the position: ";
        cin >> position;
        insertAtPosition(&head, data, position);
        break;
       }

    case 4:
    {
        cout << "Insert at a position,indexing node from 0 to N-1:" << endl;
        int data, position;
        cout << "Enter the data: ";
        cin >> data;
        cout << "Enter the position: ";
        cin >> position;
        insertAtPosition2(&head, data, position);

        break;
    }
```

```
        default:
         {
             cout << "Invalid choice." << endl;
             break;
         }
         }
    }
  return 0;
}
```

Space Complexity

Input Space:

1. Parameter – Only Definition

The input space refers to the space required for the input parameters of the function.

In insertAtPosition2, the inputs are:

- ***Node ** head: A pointer to a pointer to the head of the list.***
- ***int data: The data to be inserted into the new node.***
- ***int position: The position at which the new node should be inserted.***

The input space for these parameters is constant, i.e., O(1), because

the space required does not depend on the size of the linked list.

2. Full – Data Definition (sometimes called "inclusive" input size)

• Same three parameters (Θ(1)).

• Plus all nodes that can be reached through *head (the actual linked-list data already residing in memory).

• If the list has n nodes, its size is Θ(n).

• Complexity: Θ(n).

This more common definition includes the entire data structure pointed to by the parameters. If the linked list has **`n` nodes** before the insertion, the space needed to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

Auxiliary Space:

$\boldsymbol{Auxiliary\ space\ refers\ to\ the\ extra\ space\ or\ temporary\ space\ used\ by\ the\ algorithm.}$

$\boldsymbol{In\ insertAtPosition2, the\ auxiliary\ space\ includes}$:

- $\boldsymbol{Node\ *newNode: A\ pointer\ to\ the\ new\ node\ being\ created. This\ requires\ constant\ space, O(1)}$.
- $\boldsymbol{Node\ *current: A\ pointer\ used\ to\ traverse\ the\ list. This\ also\ requires\ constant\ space, O(1)}$.
- $\boldsymbol{int\ i: An\ integer\ used\ as\ a\ loop\ counter. This\ requires\ constant\ space, O(1)}$.

Auxiliary space is the extra working memory an algorithm uses, not counting the input. Let's examine the variables created inside this function:

1. **`Node *newNode`**: A single pointer is declared, which takes up a constant amount of space.
2. **`malloc(sizeof(Node))`**: Memory is allocated for exactly **one** new `Node`. The size of a node is fixed and does not change based on the list's length.
3. **`Node *current` and `int i`**: Inside the `else` block, a pointer and an integer are created to help traverse the list. These also occupy a constant amount of space.

Although the function's `while` loop iterates through the list (which impacts its **time complexity**), it only ever uses the same few variables (`newNode`, `current`, `i`) and allocates memory for just one node. The total amount of extra memory is fixed.

Therefore, the auxiliary space complexity is **Θ(1)**.

1. Auxiliary Space Complexity (extra space used by the algorithm itself, excluding the input)

   • One dynamically allocated Node object (newNode) → Θ(1).

   • A constant number of pointer/int locals (current, i, etc.) → Θ(1).

   • No recursion or additional dynamic structures.

   • Complexity: Θ(1).

Algorithm

$INSPOS2(START, ITEM, POS)$

$\left((Node *)malloc\left(sizeof(Node)\right) \text{ is availability of space for insertion of Data,} \\ \text{hence , lets name it } AVAIL.\right)$

1. $[ALLOCATE\ NEW\ NODE]$ $Set\ NEW := AVAIL.$

2. $[MEMORY\ ALLOCATED?]$ $if\ NEW = NULL, then:$

$Write:$ `` $Error: Memory\ Allocation\ Failed.$ `` $and\ EXIT$.

$[End\ of\ If\ structure.]$

3. $Set\ INFO[NEW] := ITEM[Copies\ new\ data\ into\ new\ node].$

4. $[USER\ INPUT\ POS(POSITION)\ GIVEN\ 0(0^{th}\ NODE)\ ]if\ POS = 0\ , then:$

$Set\ LINK[NEW] := START\ [LINK\ of\ NEW\ node\ will\ get\ connected\ to\ ADDRESS\ of\ First\ NODE\ stored\ in\ START]$

$Set\ START := NEW\ [START\ will\ now\ point\ to\ NEW\ node]$

$Return$

$[END\ of\ If\ structure]$

5. $[\ LIST\ is\ Empty?\ ]\ If\ Head = NULL\ , then:$

$Write:$ `` $Error: LIST\ is\ EMPTY.$ `` $, Return\ and\ EXIT$.

$[End\ of\ If\ structure.]$

6. $[Position\ LESS\ THAN\ 0\ ?\ ]\ if\ POS < 0\ , then:$

$Write:$ `` $Error: INVALID\ Position.$ `` $, Return\ and\ EXIT$.

$[When\ Position > 0\ , and\ List\ Is\ Not\ Empty]$

$ELSE$

$Set\ PTR := START\ [PTR\ is\ a\ pointer\ which\ is\ assigned\ to\ Head]$
$Set\ COUNT := 1\ [Count\ is\ an\ integer\ to\ count\ number\ of\ nodes]$

$Repeat\ while\ COUNT < (POS - 1)\ \ AND\ PTR\ \neq NULL:$
$Set\ PTR := LINK[PTR]\ [Updates\ PTR.]$
$Set\ COUNT := COUNT + 1\ [Updates\ COUNT.]$

$[END\ of\ Loop]$

7. $[\ PTR\ is\ NULL?\ ]\ if\ PTR = NULL\ , then:$

$Write:$ `` $Error: INVALID\ Position.$ `` $, Return\ and\ EXIT$.

$[When\ PTR\ is\ not\ NULL]$

$ELSE$

$Set\ LINK[NEW] := \ LINK[PTR]\ [LINK\ of\ NEW\ node\ is\ set\ to\ Updated\ LINK\ of\ PTR]$

$Set\ \ LINK[PTR] := NEW\ \left[\begin{array}{l} LINK\ of\ PTR\ is\ set\ to\ address\ of\ NEW\ node, this\ will\ move\ the\ NEW\ node \\ , next\ to\ desired\ node\ and\ put\ the\ new\ node\ to\ the\ user\ input\ position\ . \end{array}\right]$

$[End\ of\ If\ structure.]$

$Note: 5th\ Step\ is\ optional, as\ 4rth\ Step\ put\ the\ node\ in\ first\ position, hence\ no\ need\ to\ check\ the\ Head\ or\ Start\ is\ Nullptr\ ,$

$i.e.\ List\ Is\ Empty.$